

Logging using SLF4J

EXERCISE 1 BLUEPRINT & TECHNICAL REFERENCE MANUAL

Exercise Objective:

- Understand the separation between log facades (SLF4J) and log implementations (Logback).
- Configure dependencies properly within an Apache Maven infrastructure pipeline.
- Initialize class-scoped runtime loggers and output explicit Error and Warning level events.

Step 1: Dependency Management Configuration

To use SLF4J with Logback, you must provide both the unified abstraction layer API and the concrete executable framework engine inside your project's `pom.xml` file:

```
<dependencies>
  <!-- SLF4J API Facade Core Definition -->
  <dependency>
    <groupId>org.slf4j</groupId>
    <artifactId>slf4j-api</artifactId>
    <version>1.7.30</version>
  </dependency>

  <!-- Logback Classic Native Binding Framework Engine -->
  <dependency>
    <groupId>ch.qos.logback</groupId>
    <artifactId>logback-classic</artifactId>
    <version>1.2.3</version>
  </dependency>
</dependencies>
```

Step 2: Complete Implementation Code

The code below sets up a clean, production-ready logging class. It generates a thread-safe static logger instance to safely capture operations across application contexts.

```

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class LoggingExample {

    // Create a single thread-safe, class-bound logger subsystem reference
    private static final Logger logger = LoggerFactory.getLogger(LoggingExample.class);

    public static void main(String[] args) {

        // 1. ERROR Level: Applied when operation branches completely fail or throw fatal fa
        logger.error("This is an error message");

        // 2. WARN Level: Applied to record edge conditions, performance bottlenecks, or dep
        logger.warn("This is a warning message");

    }
}

```

Understanding the Logging Levels Used

ERROR Level (logger.error)

Indicates severe operational failures. This level requires immediate attention or notification routing because a critical path (e.g., database connection drops, missing files, failed remote api actions) was disrupted.

WARN Level (logger.warn)

Flags unexpected behaviors that do not stop the application. Examples include using fallback parameters, encountering long processing times, or receiving invalid user input that was handled safely.